

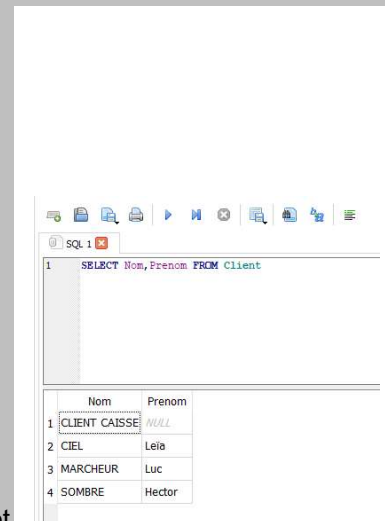
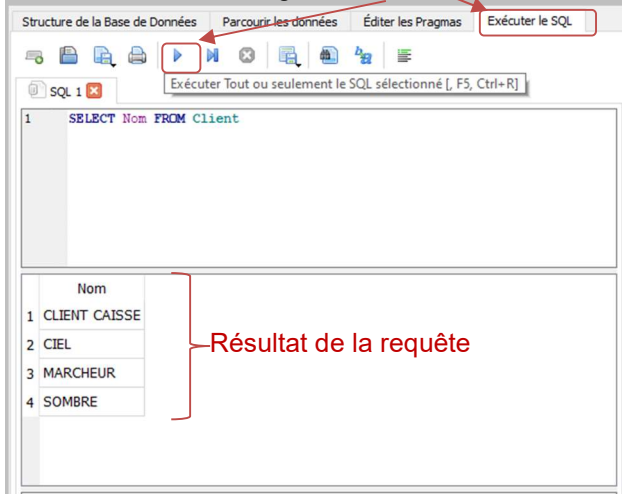
3. Les requêtes SQL :

L'instruction de base pour l'interrogation d'une base de données en SQL est constituée du mot-clé **SELECT** suivi du mot-clé **FROM**.

✚ **SELECT** permet de sélectionner les attributs dont il faut afficher les valeurs. Le caractère * permet d'afficher les valeurs de tous les attributs.

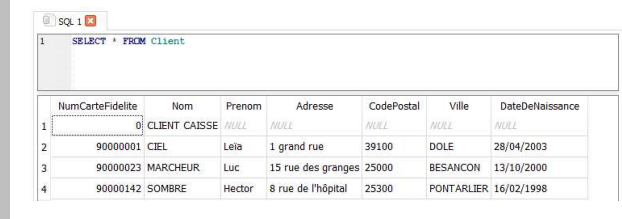
✚ **FROM** permet de sélectionner la relation à explorer.

Exemple : on souhaite afficher le nom de tous les clients : on exécutera l'instruction **SELECT Nom FROM Client** dans l'onglet Exécuter le SQL.



On peut afficher plusieurs colonnes, exemple si l'on veut le nom et prénom, il suffit de séparer d'une virgule.

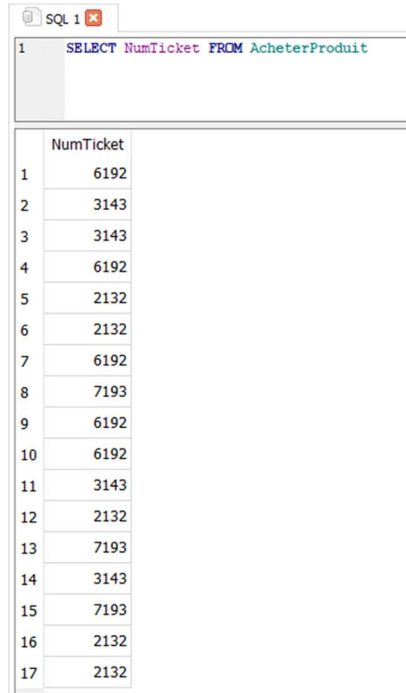
On peut lui faire afficher toute la table avec *.



Donner l'instruction pour afficher les numéros de ticket à partir de la table AcheterProduit.

Réponse :

SELECT NumTicket FROM AcheterProduit



On constate qu'il y a plusieurs fois la même valeur. Il est possible de ne pas afficher les doublons à l'aide du mot clé **DISTINCT**.

Tester l'instruction **SELECT DISTINCT NumTicket FROM AcheterProduit** pour afficher les numéros de ticket à partir de la table AcheterProduit.

```
1 SELECT DISTINCT NumTicket FROM AcheterProduit
```

	NumTicket
1	6192
2	3143
3	2132
4	7193

On peut aussi trier ces valeurs à l'aide du mot clé **ORDER BY** (pour l'ordre décroissant, il faut rajouter **DESC**)

Tester l'instruction **SELECT DISTINCT NumTicket FROM AcheterProduit ORDER BY NumTicket** pour afficher les numéros de ticket à partir de la table AcheterProduit du plus petit au plus grand.

```
1 SELECT DISTINCT NumTicket FROM AcheterProduit ORDER BY NumTicket
```

	NumTicket
1	2132
2	3143
3	6192
4	7193

Télécharger la base de données suivantes : [Communes.db](https://www.communes.fr/)

	idVille	nomVille	population	nbElusMunicipaux	nbInscritsListesElectorales	exprimesPremierTour	exprimesDeuxiemeTour
	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre	Filtre
1	1001	L'Abergement-Clémenciat	776	15	598	341	NULL
2	1002	L'Abergement-de-Varey	248	11	212	188	NULL
3	1004	Ambérieu-en-Bugey	14035	33	8101	4585	4368
4	1005	Ambérieux-en-Bombes	1689	19	1186	479	NULL
5	1006	Ambléon	111	11	99	66	NULL
6	1007	Ambronay	2726	23	1883	1191	1304
7	1008	Ambutrix	752	15	551	337	NULL
8	1009	Andert-et-Condon	330	11	269	176	NULL
9	1010	Anglefort	1115	15	733	520	NULL

1. Quelle requête permet d'afficher tous les noms de commune ?

Réponse :

```
1 SELECT nomVille FROM Communes
```

	nomVille
1	L'Abergement-Clémenciat
2	L'Abergement-de-Varey
3	Ambérieu-en-Bugey
4	Ambérieux-en-Bombes
5	Ambléon
6	Ambronay
7	Ambutrix
8	Andert-et-Condon
9	Anglefort

2. Quelle requête permet d'afficher tous les noms de commune et leur population ?

Réponse :

```
1 SELECT nomVille, population FROM Communes
```

	nomVille	population
1	L'Abergement-Clémenciat	776
2	L'Abergement-de-Varey	248
3	Ambérieu-en-Bugey	14035
4	Ambérieux-en-Bombes	1689
5	Ambléon	111
6	Ambronay	2726
7	Ambutrix	752
8	Andert-et-Condon	330
9	Anglefort	1115

3. Quelle requête permet d'afficher tous les noms de commune et leur population, classé par ordre croissant de population ?

Réponse :

SQL 1

```
1 SELECT nomVille, population FROM Communes ORDER BY population
```

	nomVille	population
1	Ambbléon	111
2	L'Abergement-de-Varey	248
3	Andert-et-Condon	330
4	Ambutrix	752
5	L'Abergement-Clémenciat	776
6	Anglefort	1115
7	Ambérieux-en-Bombes	1689
8	Ambronay	2726
9	Ambérieu-en-Bugey	14035

4. Quelle requête permet d'afficher tous les noms de commune et leur population, classé par ordre décroissant d'élus municipaux ?

Réponse :

SQL 1

```
1 SELECT nomVille, population FROM Communes ORDER BY nbElusMunicipaux DESC
```

	nomVille	population
1	Ambérieu-en-Bugey	14035
2	Ambronay	2726
3	Ambérieux-en-Bombes	1689
4	L'Abergement-Clémenciat	776
5	Ambutrix	752
6	Anglefort	1115
7	L'Abergement-de-Varey	248
8	Ambbléon	111
9	Andert-et-Condon	330

Faire une capture pour chaque question.

Une restriction est une sélection de lignes d'une relation, sur la base d'une condition à respecter, définie à la suite du terme **WHERE**. Cette condition peut être une combinaison de comparaisons (=, <>, >, >=, <, <=) à l'aide de **AND**, de **OR** et de **NOT** (attention donc aux parenthèses dans ce cas).

Attention : Pour les comparaisons de chaînes de caractères, il est important de faire attention à la casse. Par définition, un "a" est donc différent d'un "A". Pour remédier à ce problème, il existe les fonction upper() et lower() pour transformer une chaîne en respectivement majuscule et minuscule.

Proposer une requête pour afficher les produits dont le prix est supérieur à 10€.

Faire une capture du résultat.

Réponse :

SQL 1

```
1 SELECT NomProduit FROM Produit WHERE PrixTTC>10
```

	NomProduit
1	Huile 5W30 3l

Proposer une requête pour afficher les produits dont le prix est supérieur à 5€ et en stock.
 Faire une capture du résultat.

Réponse :

SQL 1

```
1 SELECT NomProduit FROM Produit WHERE PrixTTC>5 AND EnStock=1
```

	NomProduit
1	Galette rois
2	Huile 5W30 3l
3	Mgrt Canard
4	Buche citron

A l'aide de la base de données Communes.db précédente :

1. Quelle requête permet d'afficher le nom des communes dont le nombre d'élus municipaux est strictement inférieur à 10 ?

Réponse :

Il n'y en pas

SQL 1

```
1 SELECT nomVille FROM Communes WHERE nbElusMunicipaux<10
```

L'exécution s'est terminée sans erreur.
 Résultat : 0 enregistrements ramenés en 5ms
 À la ligne 1 :
 SELECT nomVille FROM Communes WHERE nbElusMunicipaux<10

2. Quelle requête permet d'afficher le nom des communes dont le nombre d'élus municipaux est strictement supérieur à 10 ET dont la population est inférieure ou égale à 150 ?

Réponse :

SQL 1

```
1 SELECT nomVille FROM Communes WHERE nbElusMunicipaux>10 AND population<=150
```

	nomVille
1	Ambléon

3. Quelle requête permet d'afficher le nombre d'élus municipaux et la population de la ville de « anglefort » ? (attention à la casse)

Réponse :

SQL 1

```
1 SELECT nbElusMunicipaux,population FROM Communes WHERE upper(nomVille)="ANGLEFORT"
```

	nbElusMunicipaux	population
1	15	1115

Une **donnée manquante** en SQL est repérée par un **NULL**. Il y a plusieurs raisons, bonnes ou mauvaises, pour avoir des données manquantes, et il est parfois utile de tester leur présence. Pour cela, nous allons utiliser le terme **IS NULL** comme condition.
Au contraire, si l'on veut uniquement les données pour lesquels l'information est présente, nous devons utiliser la négation avec **IS NOT NULL**.

A l'aide de la base de données Communes.db précédente :

1. Quelle requête permet d'afficher le nom des communes dans lesquelles il y a eu un deuxième tour ?

Réponse :

```
SQL 1
1 SELECT nomVille FROM Communes WHERE exprimesDeuxiemeTour IS NOT NULL
```

	nomVille
1	Ambérieu-en-Bugey
2	Ambronay

2. Quelle requête permet d'afficher le nom des communes dont la population est supérieure à 5 000 habitants et dans lesquelles il y a eu un deuxième tour ?

Réponse :

```
SQL 1
1 SELECT nomVille FROM Communes WHERE population>5000 AND exprimesDeuxiemeTour IS NOT NULL
```

	nomVille
1	Ambérieu-en-Bugey

L'opérateur **like** permet de rechercher les **valeurs contenant une partie seulement de la chaîne de caractères**. Le caractère % représente une suite de caractères, éventuellement nulle.

A l'aide de la base de données Communes.db précédente :

1. Quelle requête permet d'afficher le nom des communes dont le nom commence par Roncherolles ?

Réponse : Il n'y en a pas

```
SQL 1
1 SELECT nomVille FROM Communes WHERE upper(nomVille) like "RONCHEROLLES%"
```

L'exécution s'est terminée sans erreur.
Résultat : 0 enregistrements ramenés en 3ms
À la ligne 1 :
SELECT nomVille FROM Communes WHERE upper(nomVille) like "RONCHEROLLES%"

2. Quelle requête permet d'afficher le nom des communes dont le nom contient le mot « -en- » ?

Réponse :

```
SQL 1
1 SELECT nomVille FROM Communes WHERE upper(nomVille) like "%-en-%"
```

	nomVille
1	Ambérieu-en-Bugey
2	Ambérieux-en-Bombes

La fonction d'agrégation **count()** permet de compter le nombre d'enregistrement dans une table.

La fonction **sum(attribut)** permet de faire la somme des valeurs non nulles de l'attribut passé en paramètre.

La fonction **avg(attribut)** permet de faire la moyenne des valeurs de l'attribut passé en paramètre.

La fonction **min(attribut)** permet d'afficher la valeur minimale de l'attribut passé en paramètre.

La fonction **MAX(attribut)** permet d'afficher la valeur maximale de l'attribut passé en paramètre.

A l'aide de la base de données Communes.db précédente :

1. Quelle requête permet d'afficher le nombre total d'élus de toutes les communes ?

Réponse :

```
SQL 1
1 SELECT sum(nbElusMunicipaux) FROM Communes
```

	sum(nbElusMunicipaux)
1	153

2. Quelle requête permet d'afficher la moyenne du nombre d'habitants dans les communes ?

Réponse :

```
SQL 1
1 SELECT avg(population) FROM Communes
```

	avg(population)
1	2420.2222222222

3. Quelle requête permet d'afficher le nom de la ville avec la population la plus grande pour laquelle il n'y a pas de deuxième tour ?

Réponse :

SQL 1

```
1 SELECT nomVille, max(population) FROM Communes WHERE exprimesDeuxiemeTour IS NULL
```

	nomVille	max(population)
1	Ambérieux-en-Bombes	1689

Dans un modèle relationnel, une clé étrangère référence une clé primaire venant d'une autre table. Cela s'appelle une jointure. Grâce à celle-ci, on peut récupérer des données provenant de plusieurs tables.

La syntaxe est : table1 **JOIN** table2 **ON** table1.champ1 = table2.champ2

La jointure se positionne après les clauses **SELECT** et **FROM**.

Reprenons notre base de données ticket.db :

1. Quelle requête permet d'obtenir le nom de tous les articles achetés ainsi que leurs quantités par le ticket 3143.

Réponse :

SQL 1

```
1 NomProduit, Quantite FROM Produit JOIN AcheterProduit ON Produit.CodeProduit=AcheterProduit.CodeProduit WHERE NumTicket=3143
```

	NomProduit	Quantite
1	Pain épice miel	3
2	Baguette Rust	1
3	Mgrt Canard	1
4	2kg orange jus	1

2. Quelle requête permet d'obtenir le nom de tous les articles achetés ainsi que leurs quantités par Luc Marcheur.

Réponse :

SELECT NomProduit, Quantite **FROM** Produit
JOIN AcheterProduit **ON** Produit.CodeProduit = AcheterProduit.CodeProduit
JOIN Ticket **ON** AcheterProduit.NumTicket = Ticket.NumTicket
JOIN Client **ON** Client.NumCarteFidelite = Ticket.NumCarteFidelite
WHERE upper(Nom) = 'MARCHEUR' **AND** upper(Prenom) = 'LUC';

SQL 1

```
1 SELECT NomProduit, Quantite
2 FROM Produit JOIN AcheterProduit ON Produit.CodeProduit=AcheterProduit.CodeProduit
3 WHERE NumTicket=(
4 SELECT NumTicket
5 FROM Ticket JOIN Client ON Ticket.NumCarteFidelite = Client.NumCarteFidelite
6 WHERE upper(Client.Nom)="MARCHEUR" AND upper(Client.Prenom)="LUC")
```

	NomProduit	Quantite
1	Galette rois	1
2	Baguette Rust	1
3	1l Jus Pom. Brt	2
4	Lave glace E 5l	1
5	Lave glace H 5l	1

L'exécution s'est terminée sans erreur.
Résultat : 5 enregistrements ramenés en 5ms
À la ligne 1 :
SELECT NomProduit, Quantite
FROM Produit JOIN AcheterProduit ON Produit.CodeProduit=AcheterProduit.CodeProduit
WHERE NumTicket=(
SELECT NumTicket
FROM Ticket JOIN Client ON Ticket.NumCarteFidelite = Client.NumCarteFidelite
WHERE upper(Client.Nom)="MARCHEUR" AND upper(Client.Prenom)="LUC")